SANS COULEURS

Document explicatif du code

Jeu de plateforme à deux joueurs — Phaser 3, JavaScript, Tiled

Lilian Cornet — Octobre 2026

github.com/zaderlyl/sans-couleurs

Présentation du jeu

Sans Couleurs est un jeu de plateforme 2D en pixel art pour **deux joueurs**, développé avec Phaser 3 (JavaScript, sans outil de build). C'est l'histoire de la vie d'un personnage qui a mal vécu son enfance : un monde gris, des enfants qui le trouvent « différent », des mots qui pèsent. Un joueur joue à la souris, l'autre au clavier ; on n'avance qu'ensemble (le train ne part que si les deux agissent dessus, et les mots du collège ne s'allègent que si les deux restent côte à côte). Le jeu a trois niveaux (le début, l'enfance, le collège) précédés d'une page d'accueil.

Les commandes

	Joueur souris	Joueur clavier
Marcher	clic dans le vide	flèches gauche / droite
Interagir	clic sur l'élément (PNJ, gare)	touche E dans la gare ; se placer à côté de l'autre (niveau 3)
Tirer (niveaux 1 et 2)	Ctrl ou Cmd + clic	barre espace

Lancer le jeu

Le jeu tourne dans le navigateur. Il faut le servir avec un petit serveur local (les navigateurs bloquent les modules JavaScript en `file://`) : `python3 -m http.server 8000` dans le dossier du jeu, puis ouvrir `http://localhost:8000`.

Organisation du code

Le code est écrit simplement, dans l'esprit des tutoriels de la matière : **une scène Phaser par fichier**, les variables et les fonctions ont des noms en français, et chaque bloc est précédé d'un commentaire qui explique pourquoi il est là. Les trois niveaux héritent d'une classe de base commune (`niveau.js`).

Fichier (dossier src/)	Rôle
`index.js`	configuration Phaser, liste des scènes, lancement
`accueil.js`	page d'accueil (titre + boutons)
`niveau.js`	classe de base : carte, deux joueurs, caméras, tir, décor
`niveau1.js, niveau2.js, niveau3.js`	les trois niveaux (ce qui est propre à chacun)
`camera.js`	la caméra : un seul écran qui suit les deux joueurs
`tir.js, cibles.js`	le tir, les cibles, les timers
`phrases.js, pastilles.js`	le niveau 3 : les mots à porter à deux, les pastilles
`gare.js, passage.js`	les éléments interactifs et les changements de niveau
`pnj.js, dialogue.js, mots_a_placer.js`	les personnages et les dialogues à trous
`decor.js, textes_de_zone.js, ecran_texte.js`	l'ambiance (herbe, tunnel, textes)
`chute.js`	tomber dans le vide (bords du monde)
`sons.js`	musique de fond et bruitages (Web Audio)

PARTIE 1 DE LA GRILLE

Création du jeu avec Phaser

Dans le jeu. La page `index.html` charge Phaser depuis un CDN et le module `src/index.js`, qui configure le jeu et le lance. Le personnage s'affiche dès le niveau 1.

index.html

```
<div id="game-container"></div>
<script src="https://cdnjs.cloudflare.com/ajax/libs/phaser/3.70.0/phaser.min.js"></script>
<script type="module" src="src/index.js"></script>
```

src/index.js

```
var config = {
  type: Phaser.AUTO,
  parent: "game-container", // le <div> de index.html
  backgroundColor: "#000000",
  pixelArt: true, // rendu net (pas de lissage), c'est du pixel art
  scale: {
    // le jeu occupe toute la fenetre et se redimensionne avec elle
    mode: Phaser.Scale.RESIZE,
    width: "100%",
    height: "100%"
  },
  physics: {
    default: "arcade",
    arcade: {
      gravity: { y: 900 },
      // la physique avance a chaque image affichee : sinon le personnage
      // semble vibrer sur les ecrans 120/144 Hz
      fixedStep: false,
      debug: false // true pour voir les hitbox
    }
  },
  // la premiere scene de la liste est lancee automatiquement
  scene: [accueil, niveau1, niveau2, niveau3]
```

src/index.js

```
Promise.race([attente_police, attente_max]).finally(function () {
  // creation et lancement du jeu a partir de la configuration
  new Phaser.Game(config);
});
```

Explications

- `type: Phaser.AUTO` choisit WebGL si le navigateur le permet, sinon Canvas.
- `pixelArt: true` désactive le lissage : le pixel art reste net.
- `Phaser.Scale.RESIZE` : le jeu occupe toute la fenêtre et s'adapte quand on la redimensionne.
- La physique **Arcade** est activée avec une gravité de 900 ; `fixedStep: false` fait avancer la physique à chaque image affichée (sans cela, le personnage vibre sur les écrans 120 ou 144 Hz).
- La liste `scene` contient les quatre scènes du jeu ; la première (l'accueil) démarre toute seule.
- Le jeu n'est créé qu'une fois la police chargée (ou après 2 secondes au maximum), pour éviter des textes dans la mauvaise police.

PARTIE 2 DE LA GRILLE

Créer une map sur Tiled et l'intégrer

Dans le jeu. Les trois niveaux sont des cartes dessinées avec **Tiled** (éditeur de cartes), exportées en JSON dans `assets/maps/`. Le jeu les charge et les affiche avec un jeu de tuiles (le collège en a un second, dessiné pour lui : portes, horloge, bancs...).

src/niveau.js — preload()

```
// l'image des tuiles, la carte Tiled (au format JSON) et les personnages
this.load.image("img_tuiles", "assets/tilesets/Tileset.png");
this.load.image("img_tuiles_college", "assets/tilesets/Tileset-college.png"); // (le niveau 3 s'en sert)
this.load.tilemapTiledJSON(this.nom_carte, "assets/maps/" + this.nom_carte + ".json");
this.load.spritesheet("img_perso", "assets/sprites/characters/player.png", {
  frameWidth: 16,
  frameHeight: 16
});
```

src/niveau.js — create()

```
// --- la carte ---
this.carte = this.make.tilemap({ key: this.nom_carte });
// "Tileset" = le nom du jeu de tuiles dans Tiled, "img_tuiles" = l'image chargee
var tuiles = [this.carte.addTilesetImage("Tileset", "img_tuiles")];
// une carte peut avoir un 2e jeu de tuiles (le college : portes, horloge, banc...)
if (this.carte.tilesets.length > 1) {
  tuiles.push(this.carte.addTilesetImage("Tileset-college", "img_tuiles_college"));
}
this.tuiles = tuiles; // (retenu : decor.js en a besoin pour le calque "tunnel")

// les calques, du plus loin au plus proche (le dernier cree est dessus)
this.calque_bg = this.carte.createLayer("bg", tuiles, 0, 0);
this.calque_bg2 = this.carte.createLayer("bg2", tuiles, 0, 0);
this.calque_sol = this.carte.createLayer("sol", tuiles, 0, 0);
// toute tuile non vide du calque "sol" est solide
this.calque_sol.setCollisionByExclusion([-1]);
```

Explications

- `tilemapTiledJSON` charge la carte exportée par Tiled ; `make.tilemap` la crée dans la scène.
- `addTilesetImage("Tileset", "img_tuiles")` relie le jeu de tuiles *tel qu'il est nommé dans Tiled* à l'image chargée.
- `createLayer` dessine chaque calque dans l'ordre (du plus loin au plus proche) : décor, décor 2, puis le sol.
- `setCollisionByExclusion([-1])` rend solide toute tuile non vide du calque « sol » ; le joueur est ensuite relié à ce calque par un *collider*.
- Le point d'apparition du joueur est un objet « spawn » posé dans Tiled et retrouvé par `findObject` : on peut le déplacer sans toucher au code.
- Une carte peut avoir un **deuxième jeu de tuiles** : la liste `tuiles` est donnée à `createLayer` (le collège s'en sert pour ses portes et ses meubles).
- Les textes et les personnages sont aussi des objets Tiled : le contenu du jeu se modifie depuis la carte.



Le niveau 3 : un couloir de collège de 120 tuiles (classe 1, distributeur, banc, panneau), avec ses propres tuiles.

PARTIE 3 DE LA GRILLE

Cibles et fonction de tir

Dans le jeu. Le joueur souris tire avec **Ctrl (ou Cmd) + clic** vers l'endroit cliqué, le joueur clavier avec la **barre espace** devant lui. Une goutte colorée (rouge, vert, bleu à tour de rôle) part du joueur. Les cibles sont des taches grises : une cible touchée prend la couleur du projectile puis disparaît (le monde retrouve ses couleurs).

src/tir.js

```
// la direction : l'angle du joueur vers le point vise
var depart_x = joueur.x;
var depart_y = joueur.y - 2; // vers le milieu du corps
var angle = Phaser.Math.Angle.Between(depart_x, depart_y, cible_x, cible_y);
var distance = Math.min(Phaser.Math.Distance.Between(depart_x, depart_y, cible_x, cible_y), portee_max);

// le projectile apparait un peu devant le joueur
var projectile = scene.groupe_projectiles.create(
  depart_x + Math.cos(angle) * 6,
  depart_y + Math.sin(angle) * 6,
  "img_goutte"
);
projectile.setDepth(18); // au-dessus du decor
projectile.setTint(couleurs_tir[scene.couleur_tir]);
projectile.body.setVelocity(Math.cos(angle) * vitesse_projectile, Math.sin(angle) * vitesse_projectile);

// ou il doit eclater, et sa couleur : retenus sur le projectile lui-meme
projectile.arrivee_x = depart_x + Math.cos(angle) * distance;
projectile.arrivee_y = depart_y + Math.sin(angle) * distance;
projectile.couleur = couleurs_tir[scene.couleur_tir];
projectile.tireur = joueur; // qui a tire (les phrases du niveau 3 le demandent)

// couleur suivante : rouge, vert, bleu, rouge...
scene.couleur_tir = (scene.couleur_tir + 1) % couleurs_tir.length;
jouer_son_tir();
```

src/cibles.js

```
function toucher_cible(scene, projectile, cible) {
  cible.body.enable = false; // elle ne peut plus etre touchee
  scene.cible_a_la_position[cible.indice] = null;
  cible.setTint(projectile.couleur); // elle prend la couleur du projectile...
  projectile.destroy();
  jouer_son_cible();

  // ... puis elle grossit un peu en s'effacant, et disparait
  scene.tweens.add({
    targets: cible,
    scale: 1.4,
    alpha: 0,
    duration: 600,
    onComplete: function () {
      cible.destroy();
    }
  });

  scene.cibles_touchees = scene.cibles_touchees + 1;
  afficher_compteur(scene);
}
```

Explications

- Les projectiles sont dans un *groupe* physique avec `allowGravity: false` : ils volent droit.
- La direction vient de l'angle entre le joueur et le point visé (`Phaser.Math.Angle.Between`) ; la vitesse est obtenue avec `cos` et `sin`.
- **Collision balles / cibles** : `physics.add.overlap(projectiles, cibles, ...)` appelle `toucher_cible` ; la balle est détruite, la cible prend la couleur (`setTint`) puis disparaît, et le compteur augmente.
- Un projectile qui touche un mur du sol éclate (`collider` avec le calque de sol).
- Le délai entre deux tirs est géré par un timer (voir la partie 10).
- Le niveau 3 n'a pas de tir : il y est désactivé (`tir_desactive`), le niveau parle de poids et non de combat.

PARTIE 4 DE LA GRILLE

Ajout de niveaux

Dans le jeu. Il y a quatre scènes distinctes : l'**accueil** et trois niveaux. On passe de l'une à l'autre par les boutons de l'accueil, le **train** (niveau 1 vers 2) et un **passage** (niveau 2 vers 3), avec un fondu au noir.

src/index.js

```
// la premiere scene de la liste est lancee automatiquement
scene: [accueil, niveau1, niveau2, niveau3]
```

src/niveau1.js

```
import { creer_pastilles, mettre_a_jour_pastilles } from "./pastilles.js";

export default class niveau1 extends niveau {
  constructor() {
    // la cle de la scene, et le nom de la carte Tiled
    super("niveau1", "map-1-debut");
  }

  creer_particularites() {
    creer_pastilles(this);
    creer_cibles(this);
    creer_gare(this, { niveau_suivant: "niveau2", sortie_active: true });
  }

  mettre_a_jour_particularites(temps, delta) {
```

src/gare.js — changer_de_niveau()

```
if (gare.options.niveau_suivant != null) {
  // on demarre le niveau suivant (l'ecran est deja noir). On lui dit qu'on
  // arrive en train, avec l'ecran de texte.
  scene.scene.start(gare.options.niveau_suivant, { arrivee_en_train: true, avec_transition: true });
  return;
}
```

src/passage.js

```
function prendre_le_passage(scene, passage) {
  scene.passage_en_cours = true;
  scene.joueur_souris.body.setVelocity(0, 0);

  scene.cameras.main.fadeOut(400, 0, 0, 0);
  scene.cameras.main.once("camerafadeoutcomplete", function () {
    scene.scene.start(passage.vers); // sans "arrivee_en_train" : on arrive au point de depart du niveau
  });
}
```

Explications

- Chaque scène est une classe qui hérite de `Phaser.Scene` ; sa *clé* (`"niveau1"`...) permet de la lancer avec `this.scene.start("niveau2")`.
- Les trois niveaux partagent une classe de base `niveau` (carte, joueurs, caméras, tir) ; chaque niveau remplit seulement deux méthodes : `creer_particularites()` et `mettre_a_jour_particularites()`. C'est ce qui permet d'ajouter un niveau en quelques lignes.
- Quand on arrive en train, la scène suivante reçoit des données (`{ arrivee_en_train: true }`) lues dans sa méthode `init()`. Un piège de Phaser est géré : une scène démarrée sans données récupère celles du démarrage précédent, donc elles sont effacées après lecture.

PARTIE 5 DE LA GRILLE

Ajout d'un second joueur

Dans le jeu. Le joueur clavier est un second personnage **aux couleurs inversées**, contrôlé avec les flèches. Il est indépendant du joueur souris (sa propre physique, ses propres touches) ; ils se traversent. Il n'y a qu'un seul écran, qui suit les deux joueurs : si l'un sort du cadre, il apparaît en **pastille** au bord de l'écran.

src/niveau.js — create()

```
// --- le joueur clavier (2e personnage) ---
// meme chose que le joueur souris, mais un peu a droite, avec l'image
// inversee. Les deux joueurs se traversent : on ne declare aucune
// collision entre eux.
this.joueur_clavier = this.physics.add.sprite(depart.x + 24, depart.y, "img_perso2");
this.physics.add.collider(this.joueur_clavier, this.calque_sol);
this.joueur_clavier.setCollideWorldBounds(true);
```

src/gare.js — le train à deux

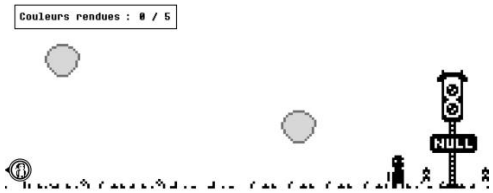
```
function verifier_le_depart(scene) {
  var gare = scene.gare;
  if (gare.souris_prete == true && gare.clavier_pret == true && peut_partir(scene) == true) {
    gare.souris_prete = false;
    gare.clavier_pret = false;
    appuyer_sur_l_icone_puis(scene, partir);
  }
}
```

src/phrases.js — les deux joueurs côte à côte allègent les mots

```
var ecart = Phaser.Math.Distance.Between(joueurs[0].x, joueurs[0].y, joueurs[1].x, joueurs[1].y);
if (ecart < distance_ensemble) {
  scene.temps_ensemble = scene.temps_ensemble + delta;
  if (scene.temps_ensemble >= delai_detache) {
}
```

Explications

- Le second personnage réutilise la même feuille d'images, mais `player_inverse.png` (couleurs inversées, créée une fois pour toutes) ; il a sa propre animation de marche.
- Aucun *collider* n'est déclaré entre les deux joueurs : ils se traversent.
- **Interactions visibles entre les deux joueurs** : le train ne part que si le joueur souris clique sur la gare **et** que le joueur clavier, dans la gare, appuie sur E (« Je ne peux pas partir sans lui. ») ; au collège, les mots que porte un joueur ne se détachent que si l'autre reste à côté de lui (distance inférieure à 40 pixels).
- `camera.js` règle la caméra : elle suit le milieu des deux joueurs et, s'ils s'éloignent trop, le plus avancé.
- Au niveau 2, c'est **le joueur clavier qui attrape les mots** qui tombent du ciel (`dialogue.js`) pour les donner au joueur souris, qui les place dans la phrase : chacun a son rôle dans chaque réponse.
- Il n'y a qu'une seule caméra ; `pastilles.js` pose un rond avec le visage du joueur au bord de l'écran, avec une flèche, tant qu'il est hors cadre.



Niveau 1 : la caméra reste unique ; le joueur souris, sorti du cadre à gauche, apparaît en pastille.

PARTIE 6 DE LA GRILLE

Ajout de son et de bruitages

Dans le jeu. Une **musique d'ambiance continue** (un fichier audio en boucle, `ambiance.mp3`) démarre au premier clic de l'accueil et ne s'arrête jamais, même quand on change de niveau. Des **bruitages** sont déclenchés par les événements : pas, atterrissage, tir, impact, cible touchée, mots placés dans un dialogue, **rire moqueur** d'un enfant après une réponse, chute dans le vide, et, au collège, la **sonnerie** qui annonce une vague de mots.

src/sons.js — la musique de fond

```
export function demarrer_la_musique() {
  var c = obtenir_le_contexte();
  c.resume(); // (le clic du joueur autorise le son)
  if (musique != null) {
    return;
  }
  musique = new Audio("assets/audio/ambiance.mp3");
  musique.loop = true;
  musique.volume = volume_musique;
  musique.play().catch(function () {
    musique = null; // le navigateur a refuse : on reessaiera au prochain clic
  });
}
```

src/sons.js — une note

```
function jouer_une_note(frequence_debut, frequence_fin, duree, volume, type) {
  var c = contexte_pret();
  if (c == null) {
    return;
  }
  var oscillateur = c.createOscillator();
  oscillateur.type = type;
  oscillateur.frequency.setValueAtTime(frequence_debut, c.currentTime);
  oscillateur.frequency.exponentialRampToValueAtTime(frequence_fin, c.currentTime + duree);

  // le volume descend vite vers 0 : le son s'eteint tout seul
  var gain = c.createGain();
  gain.gain.setValueAtTime(volume, c.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.001, c.currentTime + duree);

  oscillateur.connect(gain);
  gain.connect(c.destination);
  oscillateur.start();
  oscillateur.stop(c.currentTime + duree + 0.01);
}
```

src/sons.js — le rire moqueur (cinq « ha »)

```
export function jouer_son_rire_moqueur() {
  var hauteurs = [330, 310, 320, 285, 260];
  for (var i = 0; i < hauteurs.length; i++) {
    setTimeout(un_ha, i * 150, hauteurs[i]);
  }
}
```

src/dialogue.js — un bruitage déclenché par un événement

```
bulle.setText(replique);
jouer_son_rire_moqueur(); // le PNJ rit en repondant
```

Explications

- La musique est un objet `Audio` du navigateur avec `loop = true` : il vit en dehors des scènes, donc il continue quand on change de niveau.
- Les bruitages sont **synthétisés** avec l'API Web Audio (oscillateurs) : `jouer_une_note` crée un oscillateur qui glisse d'une fréquence à une autre, avec un volume qui retombe vers zéro. Chaque bruitage est un appel avec d'autres paramètres ; le rire enchaîne cinq « ha » (onde filtrée comme une voyelle) avec `setTimeout`.
- Au collège, la musique s'**alourdit** avec le poids porté : `peser_sur_la_musique` ralentit la lecture (donc la musique devient plus grave).
- Un navigateur interdit les sons avant un geste du joueur : la musique démarre au premier clic de l'accueil, et les bruitages restent muets (sans erreur) avant.

PARTIE 7 DE LA GRILLE

Création d'une page d'accueil

Dans le jeu. L'écran d'accueil affiche le titre et trois boutons cliquables : *Jouer* (niveau 1), *L'enfance* (niveau 2) et *Le collège* (niveau 3). Un clic lance le niveau et démarre la musique.

src/accueil.js

```
creer_bouton(texte, cle_niveau) {
  var bouton = this.add.text(0, 0, texte, {
    fontFamily: "DeltaruneExtended, monospace",
    fontSize: "32px",
    color: "#ffffff",
    backgroundColor: "#000000",
    padding: { x: 24, y: 10 }
  });
  bouton.setOrigin(0.5, 0.5);
  bouton.setInteractive({ useHandCursor: true }); // le curseur devient une main dessus

  // quand la souris entre sur le bouton : couleurs inversees (blanc / noir)
  bouton.on("pointerover", function () {
    bouton.setColor("#000000");
    bouton.setBackgroundColor("#ffffff");
  }, this);

  // quand elle en sort : couleurs normales
  bouton.on("pointerout", function () {
    bouton.setColor("#ffffff");
    bouton.setBackgroundColor("#000000");
  }, this);

  // clic sur le bouton : on lance le niveau
  bouton.on("pointerdown", function () {
    // la musique de fond demarre au premier clic (un navigateur interdit les
    // sons avant un geste du joueur)
    demarrer_la_musique();
    this.scene.start(cle_niveau);
  }, this);

  return bouton;
```

Explications

- Un bouton est un simple **texte** rendu interactif avec `setInteractive` ; les événements `pointerover`, `pointerout` et `pointerdown` gèrent le survol (couleurs inversées) et le clic.
- Le clic appelle `this.scene.start(cle_niveau)` : la scène d'accueil s'arrête et le niveau demandé démarre.
- La fonction `creer_bouton` est écrite une seule fois et appelée trois fois (un appel par bouton).



La page d'accueil.

PARTIE 8 DE LA GRILLE

Gérer les collisions avec le bord du monde

Dans le jeu. Si un joueur tombe dans le vide (là où il n'y a plus de sol), il touche le bas de la carte : il devient **rouge**, un son joue, puis il est **replacé au point de départ** du niveau. Cela marche pour les deux joueurs (sur la carte 1, on peut tomber à droite).

src/chute.js

```
export function creer_chute(scene) {
  var joueurs = [scene.joueur_souris, scene.joueur_clavier];
  for (var i = 0; i < joueurs.length; i++) {
    joueurs[i].body.onWorldBounds = true; // Phaser nous previent des collisions avec les bords
    joueurs[i].est_tombe = false; // evite de tomber plusieurs fois de suite
  }

  // l'ecouteur : corps = la hitbox qui a touche le bord ; haut, bas, gauche,
  // droite = de quel bord il s'agit
  scene.physics.world.on("worldbounds", function (corps, haut, bas, gauche, droite) {
    if (bas == false) {
      return; // seul le bord du BAS compte : on est tombe
    }
    for (var j = 0; j < joueurs.length; j++) {
      if (corps.gameObject == joueurs[j]) {
        tomber(scene, joueurs[j]);
      }
    }
  }, scene);
}
```

src/chute.js

```
function tomber(scene, joueur) {
  if (joueur.est_tombe == true) {
    return;
  }
  joueur.est_tombe = true;
  joueur.setTint(0xff0000); // le personnage devient rouge
  joueur.body.setVelocity(0, 0);
  jouer_son_chute();
  if (joueur == scene.joueur_souris) {
    scene.destination_x = null; // il ne marche plus vers sa destination
    scene.apres_arrivee = null;
  }

  // un timer simple : dans 0,8 s, le joueur revient au depart
  scene.time.delayedCall(delai_avant_retour, function () {
    var decalage = 0; // le joueur clavier revient un peu a cote du joueur souris
    if (joueur == scene.joueur_clavier) {
      decalage = 24;
    }
    joueur.setPosition(scene.point_depart.x + decalage, scene.point_depart.y);
    joueur.body.setVelocity(0, 0);
    joueur.clearTint();
    joueur.est_tombe = false;

    // les cameras se placent d'un coup sur le depart (sinon elles glisseraient de loin)
    scene.sauter_camera = true;
    scene.time.delayedCall(200, function () {
      scene.sauter_camera = false;
    }, null, scene);
  }, null, scene);
}
```

Explications

- `setCollideWorldBounds(true)` (dans `niveau.js`) empêche le joueur de sortir de la carte ; `body.onWorldBounds = true` demande à Phaser de *prévenir* quand il touche un bord.
- On écoute l'événement `"worldbounds"` du monde : le paramètre `bas` vaut `true` si c'est le bord du bas, c'est-à-dire une chute.
- `est_tombe` évite de déclencher la chute plusieurs fois de suite ; un timer ramène le joueur au départ 0,8 s plus tard.

PARTIE 9 DE LA GRILLE

Interaction avec un élément du jeu (porte)

Dans le jeu. La **gare** est l'élément interactif : quand la souris la survole, une icône « E » apparaît et le train tremble (« on peut cliquer »). Au clic, le joueur souris marche jusqu'à la gare et se tient prêt ; le joueur clavier, dans la gare, appuie sur E. Quand les deux sont prêts, **le E éclate** (animation), le train clignote, tremble, démarre, et l'écran fait un fondu vers le niveau suivant. S'il manque quelqu'un, un texte le dit. Les personnages (PNJ) fonctionnent de la même façon : une icône, un clic, un dialogue.

src/gare.js — le clic sur la gare

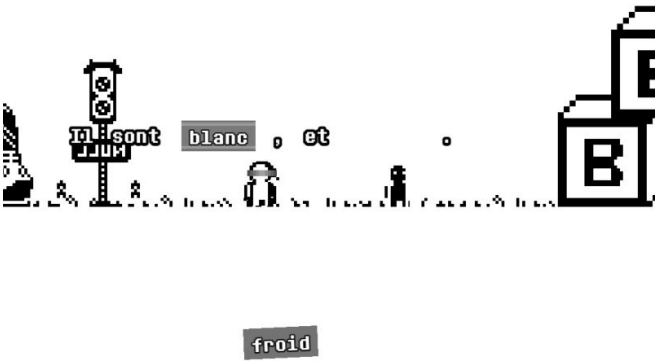
```
// clic sur la gare : le joueur souris y marche, puis le train part
gare.sprite.on("pointerdown", function (pointeur) {
  if (pointeur.event.ctrlKey == true || pointeur.event.metaKey == true) {
    return; // Ctrl / Cmd : c'est un tir
  }
  if (peut_partir(scene) == false) {
    return;
  }
  scene.aller_vers(gare.x_depart, function () {
    // le joueur souris est arrive : il est pret (tant qu'il reste dans la gare)
    gare.souris_prete = true;
    if (gare.clavier_pret == false) {
      dire_qu_il_manque_quelqu_un(scene);
    }
    verifier_le_depart(scene);
  });
});
```

src/gare.js — l'animation, puis le départ

```
function appuyer_sur_l_icone_puis(scene, suite) {
  var gare = scene.gare;
  gare.pressee = true; // (pendant ce temps, l'icone n'est pas remise en "attente")
  gare.sprite.setPosition(gare.x_sprite, gare.y_sprite); // le train arrete de trembler
  gare.icone.setVisible(true);
  gare.icone.play("anim_icone_pressee");
  gare.icone.once("animationcomplete", function () {
    gare.icone.setVisible(false);
    gare.pressee = false;
    suite(scene);
  });
}
```

Explications

- La gare est un *sprite* rendu interactif **sur ses pixels** (`zone_cliquable.js`) : seul le dessin réagit à la souris, pas le grand carré transparent autour.
- Un clic fait marcher le joueur jusqu'à la gare (`aller_vers`) ; une fois arrivé, on vérifie que le centre des **deux** joueurs est dans la zone de la gare.
- **Animation visible** : `anim_icone_pressee` joue le E qui éclate ; quand elle se termine (`animationcomplete`), la suite du voyage commence (clignotement, tremblement, départ, fondu).
- Pour le passage de la partie 4 (niveau 2 vers 3) et pour la gare, un **fondu au noir** de la caméra annonce le changement de niveau.



Autre interaction : un dialogue avec un enfant (PNJ). Les mots en bas se placent dans les trous de la phrase par un clic ou un glisser.

PARTIE 10 DE LA GRILLE

Ajout de timers pour programmer ou répéter des actions

Dans le jeu. Plusieurs timers rythment le jeu. Un **timer simple** : les cibles n'apparaissent que 1,5 s après le début du niveau ; le délai entre deux tirs ; le retour au départ 0,8 s après une chute ; un mot de la vague du collège toutes les 1,8 s. Des **timers récurrents** : toutes les 8 secondes, le gris reprend du terrain (une cible déjà effacée réapparaît et le compteur baisse d'un) ; et, au collège, **une vague de mots toutes les 25 secondes**, annoncée par une sonnerie.

src/cibles.js — timer simple et timer récurrent

```
// TIMER SIMPLE : dans 1,5 s, toutes les taches apparaissent
scene.time.delayedCall(delai_apparition, function () {
  for (var j = 0; j < positions_cibles.length; j++) {
    faire_apparaitre_une_cible(scene, j);
  }
}, null, scene);

// TIMER RECURRENT : toutes les 8 s, le gris revient
scene.time.addEvent({
  delay: delai_retour_du_gris,
  callback: le_gris_revient,
  callbackScope: scene,
  loop: true // pour toujours
});
```

src/cibles.js — ce que fait le timer récurrent

```
function le_gris_revient() {
  var scene = this; // callbackScope: scene
  // les positions ou il n'y a plus de cible
  var libres = [];
  for (var i = 0; i < scene.cible_a_la_position.length; i++) {
    if (scene.cible_a_la_position[i] == null) {
      libres.push(i);
    }
  }
  // aucune cible n'a ete touchee (ou elles ne sont pas encore apparues) : rien a faire
  if (libres.length == 0 || scene.cibles_touchees == 0) {
    return;
  }
  faire_apparaitre_une_cible(scene, Phaser.Utils.Array.GetRandom(libres));
  scene.cibles_touchees = scene.cibles_touchees - 1;
  afficher_compteur(scene);
}
```

src/phrases.js — timer récurrent : une vague de mots toutes les 25 s

```
// TIMER RECURRENT : une vague toutes les 25 s (la premiere au bout de 8 s)
scene.time.addEvent({
  delay: delai_vague,
  startAt: delai_vague - premiere_vague,
  loop: true,
  callback: function () {
    lancer_la_vague(scene);
  }
});
```

src/tir.js — timer simple : le délai entre deux tirs

```
joueur.peutTirer = false;
// le timer rend le droit de tirer dans 250 ms ("null, scene" : comme dans le tutoriel des timers)
scene.time.delayedCall(delai_entre_tirs, function () {
  joueur.peutTirer = true;
}, null, scene);
```

Explications

- `this.time.delayedCall(délai, fonction, null, scope)` exécute la fonction **une seule fois** après le délai (en millisecondes).
- `this.time.addEvent({ delay, callback, loop: true })` la répète **indéfiniment** ; `callbackScope` indique à quoi correspond `this` dans la fonction.
- Pour le tir, la propriété `joueur.peutTirer` passe à `false` au moment du tir, et le timer la remet à `true` 250 ms plus tard : c'est exactement un « délai entre deux actions ».
- Les textes qui s'écrivent lettre par lettre (`textes_de_zone.js`) utilisent aussi un timer récurrent, toutes les 70 ms.

PARTIE 11 DE LA GRILLE

Ce qui donne son identité au jeu

Au-delà de la grille, trois mécaniques portent l'histoire du jeu (la vie d'un personnage qui a mal vécu son enfance).

Les dialogues à trous

Les enfants posent une question. Chaque réponse se fait **en deux phases, à deux joueurs** : les mots **tombent du ciel** et le joueur clavier les attrape en marchant dessous ; ils arrivent alors dans la liste du joueur souris (qui a toujours plus de mots que de trous), qui les place dans les trous d'une phrase, par un clic ou un glisser-déposer. Le joueur clavier reste libre de marcher pendant tout le dialogue. Quels que soient les mots, l'enfant est mal à l'aise : il recule, dit « ... », puis une réplique qui dépend du dernier mot, et s'éloigne. Le joueur est perçu comme « différent », puis l'enfant rit. Les phrases, mots et répliques sont écrits dans Tiled (propriétés des objets).

Les mots du collège

Au collège, il n'y a ni ennemi ni tir. Toutes les 25 secondes, une sonnerie retentit et des mots arrivent **un par un**. Les mots **clairs** sont des moqueries légères (poids 1), les mots **foncés** sont des insultes (poids 2). Un mot qui touche un joueur s'accroche au-dessus de sa tête : chaque point de poids le ralentit, l'écran s'assombrit sur les bords (un voile flou) et la musique devient plus grave. On ne peut pas effacer les mots : ils ne partent que si **l'autre joueur reste à côté**, et ils se transforment alors en phrases douces (« Je suis là. »). Seul, le poids s'accumule ; à deux, il s'allège. Un joueur qui porte le poids maximum pendant 8 secondes s'effondre : fondu au noir, « Tu n'as pas pu avancer. », et les joueurs repartent de la dernière porte de classe.

src/phrases.js — accrocher un mot

```
function accrocher(scene, joueur, mot) {
  if (mot.accroche == true || poids_porte(joueur) >= maximum_portes) {
    return;
  }
  mot.accroche = true;
  mot.body.enable = false;
  scene.groupe_phrases.remove(mot); // il ne flotte plus : il est porte
  mot.setOrigin(0.5, 1);
  joueur.mots.push(mot);
  jouer_son_mot_refuse(); // un "bzzz" grave : ca pese
}
```



Le joueur clavier porte trois mots (le voile sombre s'est posé sur les bords) ; le joueur souris, hors du cadre, est signalé par une pastille à gauche.

Le train à deux

Le train ne part que si les deux joueurs agissent dessus (la souris clique, le clavier appuie sur E) : personne n'est laissé derrière, et on ne peut pas revenir en arrière. C'est l'interaction obligatoire entre les deux joueurs, qui prolonge le thème du jeu : on n'avance pas seul.

Le code complet est disponible sur le dépôt GitHub : `github.com/zaderlyl/sans-couleurs`.